温州肯恩大学
WENZHOU-KEAN UNIVERSITY

MGS 3001 WHS01
Python Programming for Business

Midterm Review

Dawei Chen

Week 7-1, 7 April 2026

Quiz Information

- Date: Thursday, April 9, 2026
- Time: 10:00 – 11:00
- Venue: CBPM A102

Format

- No make-up exam
- Closed-book written test
- 20 single-choice + 10 multiple-choice
- 1 point per question, 30 points total
- Counts for 30% of the final grade

Notes

- For multiple-choice questions, there may be 2, 3, or 4 correct answers
- Credit will be given only if all correct answers are selected
- One A4 cheat sheet is allowed
  - double-sided
  - handwritten or printed

What the quiz tests

- Reading and understanding Python code
- Identifying errors
- Debugging simple programs

Coverage

w2	3/03	Introduction to Python	Environment setup (VS Code, Jupyter Notebook), first Python script, syntax
	3/05	Python Fundamentals	Variables, core data types, operators, comments
w3	3/10	Data Structures & Control Flow	Lists, tuples, sets, dictionaries; if/else, loops (for, while, break, continue, pass)
	3/12	Functions	Functions, parameters, arguments, return, lambda expressions, list comprehensions
w4	3/19	Object-Oriented Programming	Class, object, attributes, methods
w5	3/24	Debugging & Error Handling	Try/except, trackbacks, debugging strategies
	3/26	Files & Modules	Read/write text/CSV, file operations, modules

Variables and Assignment

- `=` : Assignment operator — assigns a value to a variable
- `==` : Equality operator — checks whether two values are equal

- Variables store data values in a program. (Why we need?)
- Key Concepts
  - A variable is created when a value is assigned.
  - Naming rules for variables
  - Assigning multiple values
  - Local vs. global variables

x = 5

```
x = 5
x = x - 1
```

- Variable Assignment
  - 5 is the data value.
  - This statement assigns (binds) the value 5 to the variable x.
  - After execution, x refers to 5.
- Reassignment
  - Variables can be updated with a new value.

Python executes code sequentially, one line at a time.

Built-in Data Types

Different data types
Different classes
Different properties
Different methods

Set, check, converse data type

	Data Type	Example
Text	`str`	`"Hello, Python!"`
Numeric	`int, float, complex`	`20, 20.5, 1j`
Sequence	`list, tuple, range`	`[1, 2, 3], ("a", "b"), range(6)`
Mapping	`dict`	`{"name": "John", "age": 36}`
Set	`set, frozenset`	`{"name", "John", 36}`
Boolean	`bool`	`True, False`
Binary	`bytes, bytearray, memoryview`	`b"Hello", bytearray(5), memoryview(bytes(5))`
None	`NoneType`	`None`

Python Operators

Operator Precedence

Category	Purpose	Examples
Arithmetic Operators	Perform mathematical calculations	`+, -, *, /, %, **, //`
Assignment Operators	Assign values to variables	`=, +=, -=, *=, /=, %=, **=`
Comparison Operators	Compare two values	`==, !=, >, <, >=, <=`
Logical Operators	Combine conditional expressions	`and, or, not`
Identity Operators	Check whether two variables refer to the same object	`is, is not`
Membership Operators	Test whether a value exists in a sequence	`in, not in`
Bitwise Operators	Perform operations on binary numbers	`&, |, ^, ~, <<, >>`

Python Data Structures

Different data structures
Different classes
Different properties
Different methods

Data Structure	Examples	Ordered	Mutable	Allow Duplicates	Typical Use
List 列表	[1, 2, 3, 3]	Yes	Yes	Yes	Store and modify an ordered collection of items
Tuple 元组	(1, 2, 3, 3)	Yes	No	Yes	Represent fixed and stable groups of values
Set 集合	{1, 2, 3}	No	Yes	No	Keep only unique items and support set operations
Dictionary 字典	{“1”:1, “2”: 1}	Yes	Yes	Keys: No Values: Yes	Map keys to values for efficient access

flexible sequence → list

fixed sequence → tuple

unique values → set

key-value mapping → dictionary

Control Flow

`if` statement executes code when a condition is true.

```
username = "MGS3001"
password = False
is_active = True

if username:
  if password:
    if is_active:
      print("Login successful")
    else:
      print("Account is not active")
  else:
    print("Password required")
else:
  print("Username required")
```

`for` loop iterates over a sequence (e.g., string, list)

```
transactions = [
("Alice","Laptop",1200),
("Bob","Mouse",25),
("Alice","Keyboard",80),
("David","Laptop",1200),
("Bob","Monitor",300)
]

spending = {}

for t in transactions:
    customer = t[0]
    price = t[2]

    if customer not in spending:
        spending[customer] = price
    else:
        spending[customer] = price

print(spending)
```

`while` loop executes code as long as a condition is true

```
i = 1
while i < 6:
  print(i)
  i += 1
else:
  print("i is no longer less than 6")
```

```
for i in range(6):
    print(i)
```

Control Flow

`break`: stops the loop immediately before all iterations complete

`continue`: skips the current iteration and process to the next

`pass`: placeholder statement used when no action is required (prevents syntax errors in empty blocks)

```
fruits = ["apple", "banana", "cherry"]
for x in fruits:
  print(x)
  if x == "banana":
    break
```

```
fruits = ["apple", "banana", "cherry"]
for x in fruits:
  if x == "banana":
    continue
  print(x)
```

```
i = 1
while i < 6:
  print(i)
  if i == 3:
    break
  i += 1
```

```
i = 0
while i < 6:
  i += 1
  if i == 3:
    continue
  print(i)
```

Functions

- A function is a reusable block of code that performs a specific task
- Use `def` to define; call using `function_name(arguments)`
- Arguments
  - Positional: order matters
  - Keyword: `key=value`, order flexible
  - Default: fallback values if not provided
- Flexible Inputs
  - `*args`: multiple positional inputs (tuple)
  - `**kwargs`: multiple keyword inputs (dictionary)
- Functions can return one or multiple outputs for further use
- Lambda functions: Concise, single-expression functions for simple operations

```python
def process_scores(*scores, bonus=0):
    total = sum(scores)
    average = total / len(scores)
    final_score = average + bonus
    return average, final_score

print(process_scores(80, 90, 100, bonus=5))
```

Object-Oriented Programming (OOP)

In Python, everything is an object, with its attributes and methods.

- A class is a blueprint; an object is an instance created from that class
- OOP helps organize data and behavior together, making code more reusable, scalable, and easier to maintain
- `__init__()` method: Initializes object attributes when a new object is created
- `self` parameter: refers to the current object, used to access its attributes and methods
- Attributes
  - **Instance** attributes: unique to each object
  - **Class** attributes: shared by all objects of the class
- Methods
  - **Instance** methods: work with object data
  - **Class** methods: work with class-level data
  - **Static** methods: utility functions related to the class
- Inheritance: A child class can inherit attributes and methods from a parent class, supporting code reuse

Debugging & Error Handling

- Three types of errors
  - **Syntax error**: code cannot run
  - **Runtime error**: program crashes during execution
  - **Logical error**: code runs, but output is wrong
- How to debug systematically
  - Reproduce the error
  - Isolate the problem
  - Form a hypothesis
  - Test one change at a time
  - Verify the fix
- Error handling
  - Use `try/except` to catch runtime errors
  - Catch specific exceptions such as `ValueError` or `ZeroDivisionError`
  - `else` runs if no error occurs; finally runs no matter what
- Exam focus
  - Identify the error type
  - Understand why it happens
  - Suggest an appropriate fix

Files & Modules

- Files store data outside the program, so data can be reused, updated, and saved after the program ends
- Basic file workflow
  - open → read / write / append → close
  - Best practice: use with `open(...) as f:` so files close automatically
- Common file modes
  - `"r"`: read
  - `"w"`: write and overwrite
  - `"a"`: append
  - `"x"`: create a new file
- File paths
  - **Absolute path**: full file location
  - **Relative path**: location based on the current working directory
  - Many `FileNotFoundError` problems are actually path problems
- Modules
  - A module is a Python file with reusable code that you can import
  - Examples: `math`, `os`, `json`
- Import styles
  - `import math`
  - `from math import sqrt`
  - `import pandas as pd`

Example Questions

Q&A

CBPM B214

dchen@kean.edu

Tue & Thu 11:15 – 12:30 | Wed 14:00 – 16:30